

MODULE 9 · TROUBLESHOOTING

ConfigMap & Secret Problems

Right status, wrong assumption

Most ConfigMap/Secret failures are silent, not loud

A missing ConfigMap or Secret reference blocks the container outright — `CreateContainerConfigError`, easy to spot. A wrong key marked `optional: true`, a mismatched mount path, or a value that hasn't propagated yet all leave the Pod Running. Nothing in `describe` tells you; only the app's own behavior does.

Four failure modes cover almost every case: missing object, wrong key (required vs optional), mount-path/key mismatch, and a stale value after an update.

BY THE END YOU CAN

- Read `CreateContainerConfigError` back to the exact missing name or key
- Explain what `optional: true` actually changes
- Tell a mount mismatch from a stale value — both look identical from `get pod`

Ask the questions in order



The first two questions fail **loud** — `CreateContainerConfigError`, before the container ever starts. The third fails **quiet** — the Pod reports Running and the only evidence is in logs or exec.

What the status is telling you

STATUS	COMMON CAUSE	GO-TO COMMAND
CreateContainerConfigError	ConfigMap/Secret referenced by <code>envFrom</code> / <code>valueFrom</code> doesn't exist	<code>describe</code> (Events)
CreateContainerConfigError	Key typo in <code>configMapKeyRef</code> / <code>secretKeyRef</code> , <code>optional</code> unset or <code>false</code>	<code>describe</code> (Events)
Running , env var empty	Same key typo, but <code>optional: true</code> suppresses the error	<code>exec -- printenv</code>
Running , file not found	Mount succeeded, but the filename on disk doesn't match what the app reads	<code>exec -- ls</code> , <code>logs</code>
Running , value unchanged	Env is a start-time snapshot; volume files resync on a delay	<code>exec</code> , then wait or restart

DIAGNOSE

Two commands, in this order

- **describe first** — Events give the exact missing name or key for `CreateContainerConfigError`
- **exec / logs second** — once the Pod is Running, describe has nothing left to say

ORDER MATTERS

Running doesn't mean correct — describe stays silent past the first two questions; only the app's own output tells you.

```
$ kubectl describe pod NAME
# Events: configmap/secret "X" not found
# Events: couldn't find key X in Y

$ kubectl exec NAME -- printenv KEY
$ kubectl exec NAME -- ls MOUNTPATH
$ kubectl logs NAME
```

Running, but the file isn't where the app looks

The Secret's only key is db-password, so the mount lays down /etc/app-secret/db-password. The app reads /etc/app-secret/password. The mount itself succeeded — the Secret exists — so nothing shows up in describe.

Fix = rename the key on disk with `volumes[].secret.items[].path`, or change what the app reads.

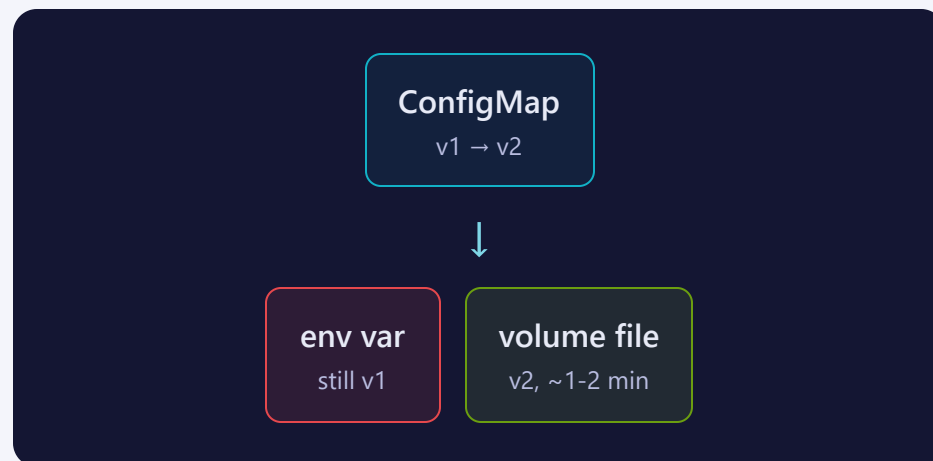
```
$ kubectl logs secret-mount-mismatch-pod
starting
FILE NOT FOUND

$ kubectl exec secret-mount-mismatch-pod -- ls
/etc/app-secret/
db-password
```

Env vars are a snapshot; mounted files are (eventually) live

Both Pods read the same ConfigMap at v1. Patch it to v2: the volume-mounted file updates within a minute or two, no restart. The env var never will — it was copied into the process environment exactly once, at container start.

```
$ kubectl rollout restart deployment/NAME  
# forces a fresh container, which re-reads the ConfigMap
```



Where people trip

- **optional: true hides a typo.** It was meant to be required — the default is false for a reason.
- **Deleting a bare Pod for a "fresh" env value.** Real workloads use `kubectl rollout restart deployment/...` so the controller replaces Pods consistently.
- **Mounting a single key with subPath.** That breaks the volume's automatic refresh — the file is never resynced.
- **Mixing data and stringData.** Pre-encoding a value for `stringData` double-encodes it — the Secret "works" but delivers garbage.

Commands to keep at hand

```
$ kubectl describe pod NAME                # Events: missing object / key
$ kubectl exec NAME -- printenv KEY          # confirm resolved env value
$ kubectl exec NAME -- ls MOUNTPATH          # confirm filename on disk
$ kubectl exec NAME -- cat MOUNTPATH/FILE    # confirm file content
$ kubectl get secret NAME -o jsonpath='{.data.KEY}' \
    | base64 -d                             # decode a stored value
$ kubectl rollout restart deployment/NAME    # force a fresh read of updated config
```

Tip: CreateContainerConfigError → describe. Running-but-wrong → exec / logs, never describe.

RECAP

ConfigMap & Secret triage in one breath

- Missing object/key = `CreateContainerConfigError`, unless `optional: true` swallows it
- Running doesn't mean correct — mount mismatches and stale values hide there
- Env vars snapshot at container start; mounted files resync on a delay
- describe for the loud failures, `exec/logs` for the quiet ones

HANDS-ON

Now run Lab 9.3
[labs/9.3/](#)

Next: [9.4 — Service Problems](#)